Server vs. Serverless: Resources and Cost Analysis of Machine Learning/Deep Learning Model Inference on Cloud Computing Services

Zixuan Zhou 59421870
zixzhou4-c@my.cityu.edu.hk
Department of Computer Science, City University of Hong Kong
Group 3

Abstract

I conduct a experiment in comparing the performance, resources and cost analysis between AWS EC2 and AWS Lambda function for ML/DL model inference. The objective of this experiment is to find the best suited solution for cloud model inference services: server-based or serverless, and provide important insight for the service provider on their own cloud architecture. The experiment collects key metrics from the combination of two models, pretrained on two different dataset, and 3 different number of request, and I find out the AWS Lambda has bad performance in terms of latency due to cold start problem. However, it provides good elastic resources allocation and become the cost-efficient choice for small request density/volume. More specifically, the lambda function requires less cost when the request density is less than 70K requests/hour than micro EC2 instances. These findings suggests that the cloud model inference service providers should identify their demands for the best possible choice for their cloud architecture.

I. Introduction

With the emerging development of Machine Learning and Deep Learning, different kinds of ML algorithms and DL models are utilized in various areas, from credit card fraud detection to AI chatbot. Many companies have integrated uses of ML/DL algorithm into their products and daily workflows. During a complete development workflow of ML/DL products, a lot of the resources are used for the data pipeline and training of the ML/DL models. However, the inference process of these models after deployment can also be extremely computational intensive, which requires stable and robust computing infrastructure. Thus, a lot of start-up AI companies choose to make use of the cloud computing resources and cooperate with large providers like Amazon and Google to support their services.

In recent years, the cloud computing services providers supports mainly two modes: server-based computing and serverless-based computing. Server-based model inference follows the traditional Infrastructure as a Service (IaaS) model. The company needs to rent a virtual machine and deploy their model and inference service on the virtual machine. Since the company takes control of the virtual machine, they can adjust their usage of OS and network accordingly. In contrast, serverless-based model inference follows the Function as a Service (FaaS) model. The cloud service provider only gives a runtime environment to the company, the company does not have the control over the OS and network usage.

Between server and serverless choices with different combinations of resources (CPU, GPU, Memory, Bandwidth etc.), it is significant for companies to find the most cost-effective choice for their services. Therefore, within this project, I want to make an experiment on server vs. serverless inference on the resources and cost analysis of ML/DL models with various resources, as well as find the tradeoff relationship between resources and amount of requests received under server and serverless circumstance using AWS EC2 [1] and AWS Lambda [2].

II. Related Work

There are some existing researches conducted on different services provided in AWS. For example, Rahul proposed his comparative analysis between AWS EC2, AWS Lambda, and AWS Sagemaker on key metrics for cloud services such as performance, scalability, cost-efficiency, load balancing, and integration. [3] Based on his comparison and contrast in between, he concludes that AWS Lambda has best efficiency,

auto-scalability and integration for event-driven model, which is best suited for quick deployment of fully-developed model. [3] Then, AWS EC2 has best flexibility and infrastructure resources control, which is best suited for a complete model development cycle requiring environment configuration management. [3] Finally, AWS Sagemaker provides robust and fully-integrated machine learning model development environment, which is user-friendly and does not require any resource management manually by user. [3] This paper provides a comprehensive comparative analysis. However, it mainly focuses on theoretical qualitative analysis instead of providing quantitative measurement from real experimens.

Next, Agarwal et al. evaluates the actual performance difference of server-based architecture (EC2) vs. serverless-based architecture (Lambda, API Gateway Dynamo DB, S3 Bucket) of E-learning platform on AWS. [4] Based on the simulated users' request load experiment, serverless-based architecture achieves 71.8% response time reduction, 80% less error rate, and 56.8% decrease of operation cost. [4] This paper provides concrete comparison experiment between server and serverless choice. However, the experiment is conducted on E-learning platform, which is more likely IO-intensive (frequent loading of educational resources) but not computing-intensive.

III. Method

Based on the previous related work, I decide to conduct the experiment on server vs. serverless comparison on computing-intensive work, which is ML/DL model inference. The whole workflow of the experiment would be divided into the following steps:

A. Preparing test dataset and ML/DL models offline

To avoid any possible bias affecting the result of our experiment, I decide to use two different models (one from ML, one from DL) to be trained on two datasets from different field.

For dataset, the first one I use is the UCI Adult Income dataset [5]. It contains demographic and employment information such as age, education, occupation, hours per week and a binary target value indicating whether the annual income is above or below 50K [5]. The second dataset is the Breast Cancer Wisconsin dataset [6]. It consists of continuous feature information derived from digitized images of breast mass such as radius, texture, perimeter and a binary label representing malignant or benign cancer. Both dataset has already been preloaded in scikit-learn package which can be used directly [6].

For the models, I decides to use one ML and one DL model respectively. The ML model I choose is XGBoost, a gradient based decision tree with limited depth for each tree, standard L1 or L2 regularization, and some other optimization techniques [7]. It utilizes gradient descent idea that the next tree will make error correction after previous tree makes a prediction, so the combined prediction from a sequence of trees will have better accuracy than normal decision tree [7]. The DL model I choose is MLP, which is an elementary DL feed forward neural network that is fully connected with both neighbour layers [8]. It utilizes back propagation algorithm to update the weight of each neuron during the training process, and try to learning enriched features based on the number of neurons in each hidden layer [8].

For all four combinations of model and dataset, they are all trained on my local computer and saved in a portable format (Json/Onnx) so that they can be reloaded in AWS without performing the training process again.

B. Writing scripts for inferencing on AWS EC2 instances (server) and AWS Lambda (serverless)

After getting the models and datasets ready, in order to run the experiment, I need to write scripts to support inference both on AWS EC2 and AWS Lambda . First and foremost, I design a predict interface for both EC2 and Lambda so that the client could test inference metrics on both computing instances following the similar command, avoiding experiment result being affected by different interfaces.

On server-based EC2 instance, I will run an HTTP server for inference. At the beginning, I will load the model (XGBoost or MLP) from the disk. Then, the server would expose the two endpoints based on

the predict interface that we use: a health check endpoint to check the current status of the service, and a prediction endpoint that takes a JSON format input and return a JSON format prediction results with latency back to the client. The latency about inference would be measured inside the server so that it will not be affected by the round trip time.

On serverless-based Lambda, I implement a handler function that could deal with inference demand whenever the function call event is happened in lambda. The event will send either the input directly or inside the body field so that the handler could parse for the input for further usage. Then, the corresponding model will be loaded (or restarted if model is already loaded) and run the inference logic through lambda. Finally, the lambda format response is sent back to the client, including header, status, and result body (including prediction and latency measurement).

C. Choosing different kinds of combination of resources (Memory, CPU) on different type of instances

I use different kinds of resources combination on both server and serverless based inference to measure the influence of resources.

For server-based EC2, I will select three different kinds of instances: t2.micro (1vCPU and 1GB of memory), t3.micro (2vCPU and 1GB of memory), and t2.medium (2vCPU and 4GB of memory). By carefully selecting different combination of CPU and memory, I could make experiment analysis on how CPU and memory affect the inference performance respectively.

For serverless-based Lambda, I cannot choose the computing instance directly. Instead, I could only decide the memory size for our function and Lambda will allocate computing resources accordingly. I choose 128Mb memory for both XGBoost and MLP so that only limited resources are wasted during the complete function deployment and inference process.

Besides, to find the relationship of instance performance and request density, I will do experiment with various inference workload size by changing the number of requests, to avoid the possible performance variability of the instance, I will repeat same configuration of the experiment multiple times to find the average performance metrics.

D. Collecting resource usage and cost information from AWS

There are two metrics that we collect from the AWS: inference latency and resources usage, so that we could calculate the inference cost based on AWS pricing dashboard and calculating prices using AWS pricing calculator or manually.

For latency metric, as mentioned earlier, all measurement will be sent back to the client as part of the prediction for both EC2 and Lambda, and the client scripts will record these metrics in a csv file for further usage. For resource metric, I will open another command window on server to record the CPU and memory usage during the EC2 experiment. For Lambda experiment, I will use the AWS cloudwatch ability to gather resource usage information directly. However, for cost metric, given the restriction that AWS Academy Lab only lasts for four hours, I might not be able to get a detailed dashboard breakdown of our cost on each resources. I have to use the pricing calculator provided by AWS or doing calculation manually for a reasonable estimation on the inference cost under each circumstance.

After gaining all measurements, we aggregate the results based on the same configuration and produce a final table as the final experiment result for further analysis.

E. Analyzing the experiment result and the tradeoff relationship between server and serverless inference

Finally, we use the collected measurements to state which instance configuration is the most preferred under various condition.

For latency analysis, the cold-start problem might affect the performance enormously, so I need to analyze both situations on Lambda: the model is not loaded, or the model has already been loaded. Both results should be compared with the result to EC2 instance to have insight analysis.

For resources analysis, some of the small instances might face resource upper limit downgrading the inference performance. I could analyze the best resources combination that reach optimal inference ability while maintaining cost efficiency.

For cost comparisons, I also analyze in two ways. The Lambda is evaluated by the cost per request (or per thousand request if necessary), while the EC2 needs to be evaluated based on the total time the server/instance is running and the request density. Thus, the inference request circumstance could directly determine the most efficient instance configuration.

To find out the tradeoff points, I can make relationship plots between the cost and request density for different AWS inference choices and try to figure out the best fitted linear/quadratic model so that the calculated tradeoff points directly represents the boundary between different configurations. This concludes the methods that I am going to use in the following experiment part.

IV. Experiment

I conduct all of my experiment (both AWS EC2 instance and Lambda function) on us-east-1 region around 8pm Hong Kong time, so the timing effect on AWS service performance is roughly controlled stable. I will show the experiment result for AWS EC2 and Lambda separately, and conduct the cost tradeoff analysis in the end.

A. AWS EC2

For the AWS EC2 experiment, I open one server instance offering inference service and recording server resources usage, and one client instance responsible for experiment request sent. As mentioned in method, the whole analysis can be separated into latency and resource aspect.

1) Latency Analysis: Firstly, for latency analysis, Figure 1 shows the histogram and cumulative density function of experiment latencies on different type of instance.

Based on the plot, the latency performance on t2.medium (2vCPU with 4GB memory) and t2.micro (1vCPU with 1GB memory) instances follows similar distribution , with most of the inference latency are at 0.2ms, with some outlier around 1ms, which is a relatively stable and robust performance. However, for letency performance on t3.micro (2vCPU with 1GB memory), compared to other two instances, a clearly distribution shift towards larger latency is demonstrated, where many of the inference latency are around 0.3ms, some inference latency distribute relatively uniform within [0.5, 1.25] ms interval, and some extremely outliers happened above 2ms. For resources changes from t2.micro to t3.micro to t2.medium, I could conclude the increased latency distribution happened in t3.micro experiment is likely due to the memory size bottleneck, which further leads to resource competition between inference request/process.

2) Resources Usage Analysis: Moreover, for resources usage during experiment, the Figure, 2, 3, 4 shows the observed value of CPU and Memory usage during the whole experiment for different instances respectively.

For CPU resources comparison, I notice that the CPU usage under t2.micro and t3.micro keeps staying around 40% for both XGBoost and MLP models. However, for t2.medium, the CPU usage for MLP models jumps to 70%, and 30% for XGBoost model, showing clearing resources requirement under different model architecture when multiple processing units are available. However, this CPU usage difference characteristic does not clearly shown inside t3.micro experiment (which is multiple processing available), further implying the existence of memory resource bottleneck.

For Memory resources comparison, MLP model has lower memory usage for both t2.medium and t2.micro than XGBoost model. However, the memory usage trend on t3.micro is in opposite direction, with higher usage for MLP model, which further enhance memory resource bottleneck implication (at least for XGBoost model) mentioned in previous analysis.

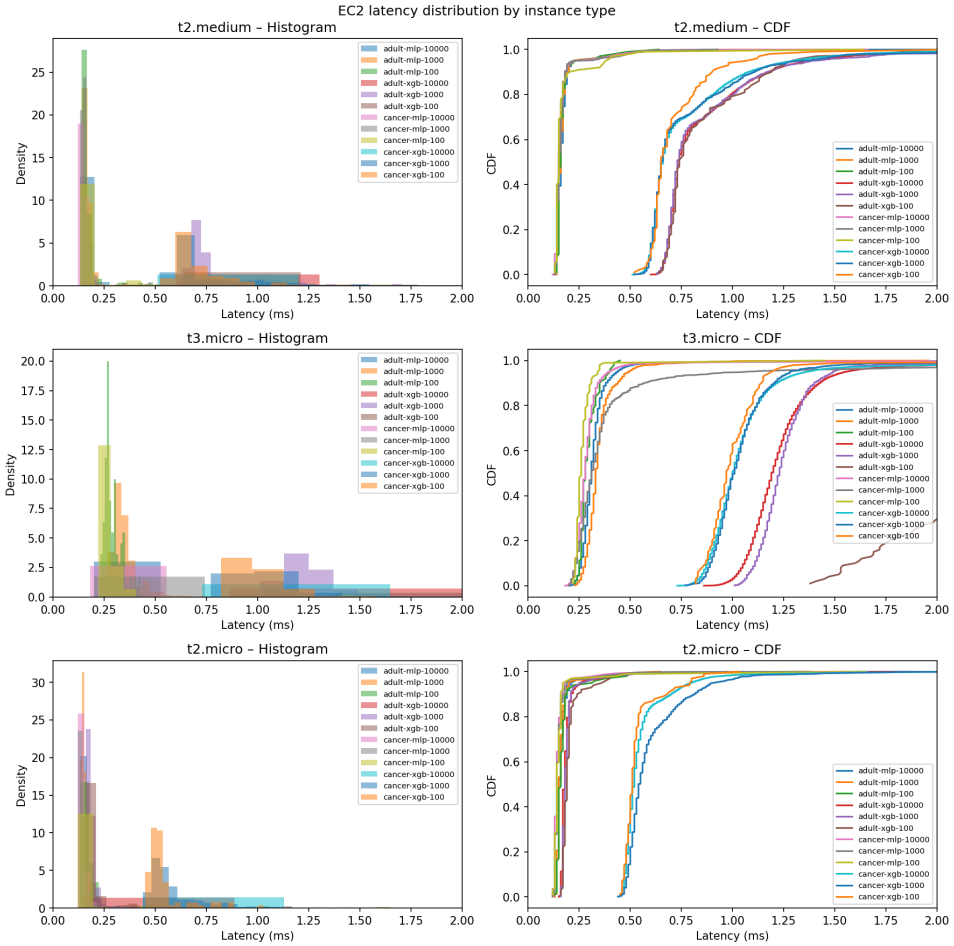

Fig. 1. Latency Histogram and Cumulative Density Function for different EC2 instances under different model, dataset, and request number combination

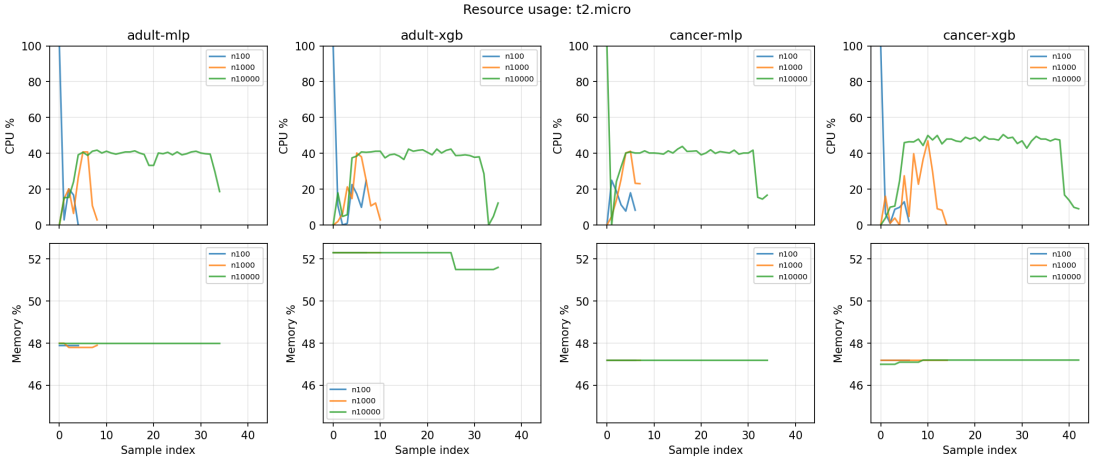

Fig. 2. CPU and Memory Usage under different model, datase and request number combination for t2.micro instance

B. AWS Lambda

For the AWS Lambda experiment, I packed all of required environment packages during inference with event-driven handler function and uploaded them to lambda function (with possible S3 bucket if the environment setup is too large). Then, I use my local computer sending request of calling lambda function

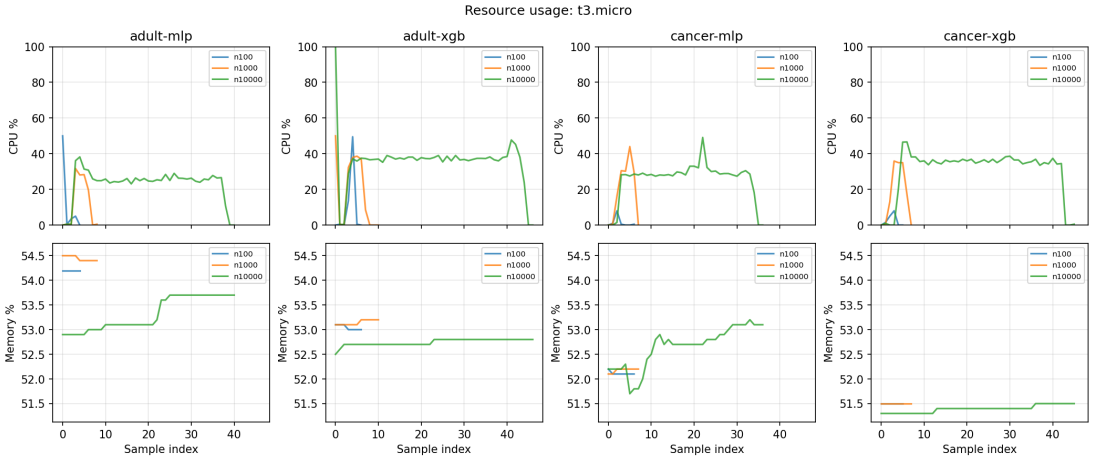

Fig. 3. CPU and Memory Usage under different model, datase and request number combination for t3.micro instance

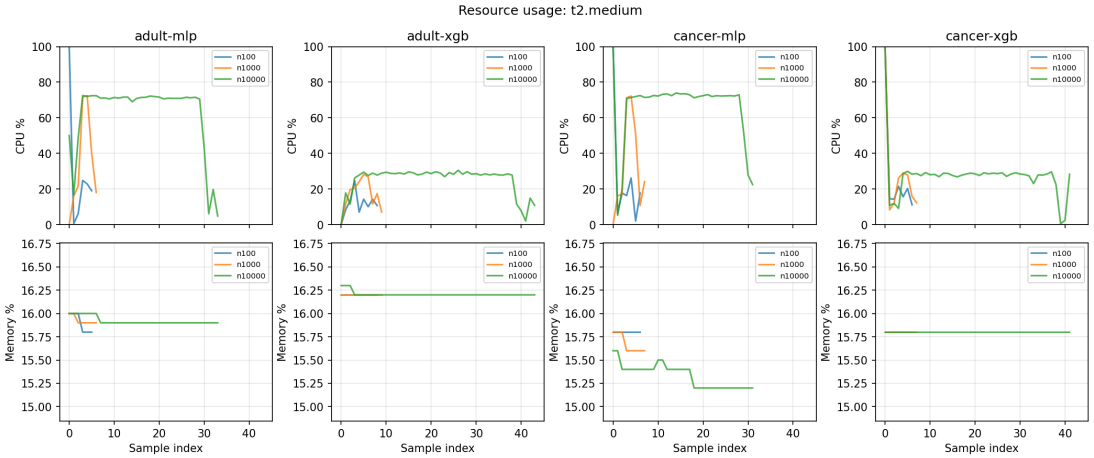

Fig. 4. CPU and Memory Usage under different model, datase and request number combination for t2.medium instance

for model inference. The latency and resources usage analysis will be discussed below.

1) Latency Analysis: As shown in the Figure 5, the majority of the inference services require around 1 ms, with some outliers between 4ms to 14ms, which is enormously higher than the inference performance of AWS EC2. The outlier latency is likely due to the cold start problem inside AWS Lambda function. However, even for inferences without cold start, the Lambda function performance is still about 4 times worse than the EC2 performance, which directly shows the latency performance weakness for AWS lambda function.

2) Resources Usage Analysis: As mentioned in method, I can only set the utilized memory size for the lambda function and the AWS will allocate the corresponding computing sources based on the requirement. The utilized memory size that i set for both MLP and XGBoost in this experiment is 128MB, which is completely utilized in both MLP and XGBoost model inferences.

C. Cost Tradeoff Analysis

Finally, I will conduct cost analysis between AWS EC2 instances and Lambda functions. The figure 6 shows the predicted (calculated) cost of inference for the experiment, based on the formula given in AWS website.

For the predicted cost within the experiment, due to the limited request size, the cost per request within EC2 instance experiments is mainly determined by the cost of the instance (t2.medium > t2.micro ≈

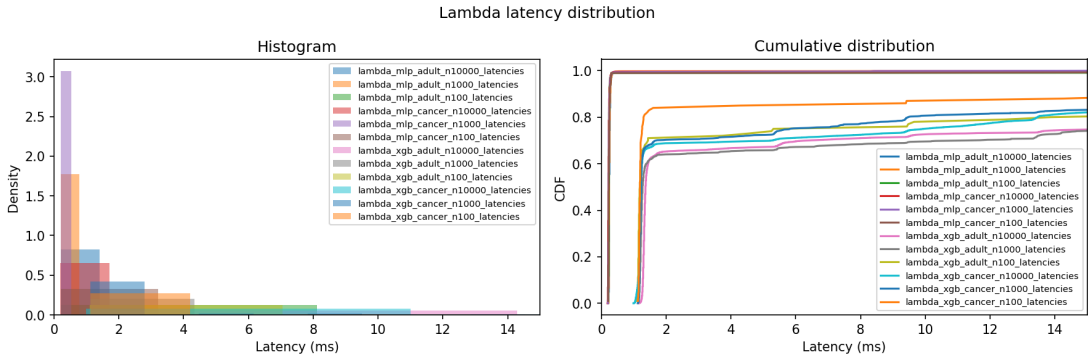

Fig. 5. Latency Histogram and Cumulative Density Function for Lambda function under different model, dataset, and request number combination

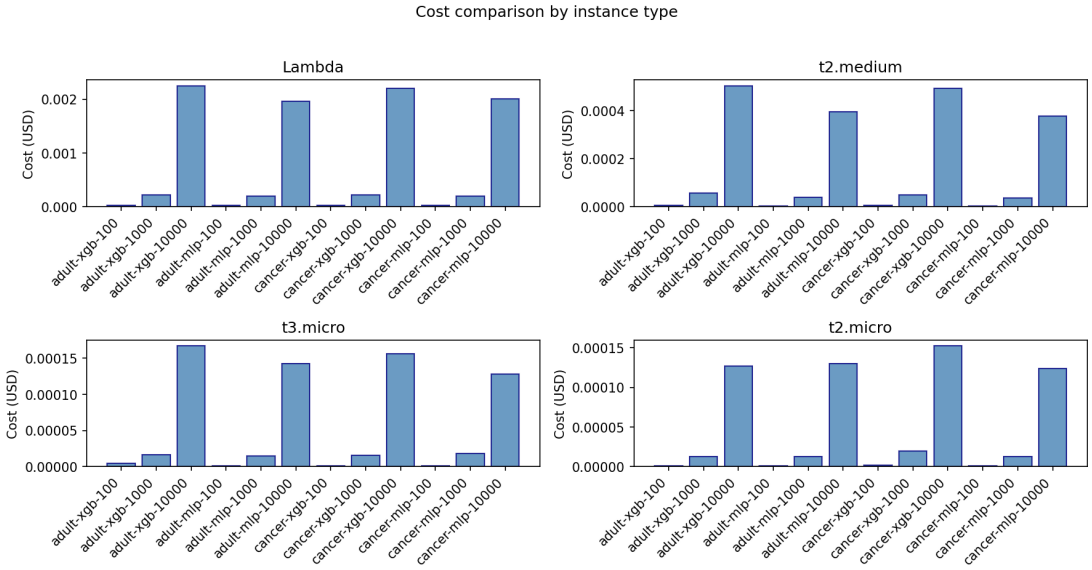

Fig. 6. Predicted cost of experiment between AWS EC2 and Lambda

t3.micro), and the cost per request for lambda experiment hugely surpassed the cost of all EC2 instance experiment.

Also, with calculation formula provided, I derive the relationship between request density and cost of AWS EC2 and Lambda when there is no congestion leading to worse inference service speed, as shown inside the Figure 7. As shown inside the plot, the cost of EC2 does not change to the request density because the time usage of instances does not change with request density. However, the cost of lambda function increases linearly (note that x-axis in the plot increases exponentially). Thus, the tradeoff point inside the plot for t2.micro and t3.micro is around 70K requests/hour, and tradeoff point for t2.medium is around 200K requests/hour, which means the user should consider use t2.micro/t3.micro for their inference services if the request density is above 70K requests/hour, and use t2.medium if the request density is above 200K request/hour.

V. Conclusion

Within this experiment, I undertake an experiment in investigating the performance, resources, and cost comparison between server-based AWS EC2 and serverless AWS Lambda for ML/DL model inference. Based on the experiment, we conclude that the latency. However, the serverless choice of AWS Lambda function can significantly decrease the cost when the request density of the inference is in small volume (eg. < 10K

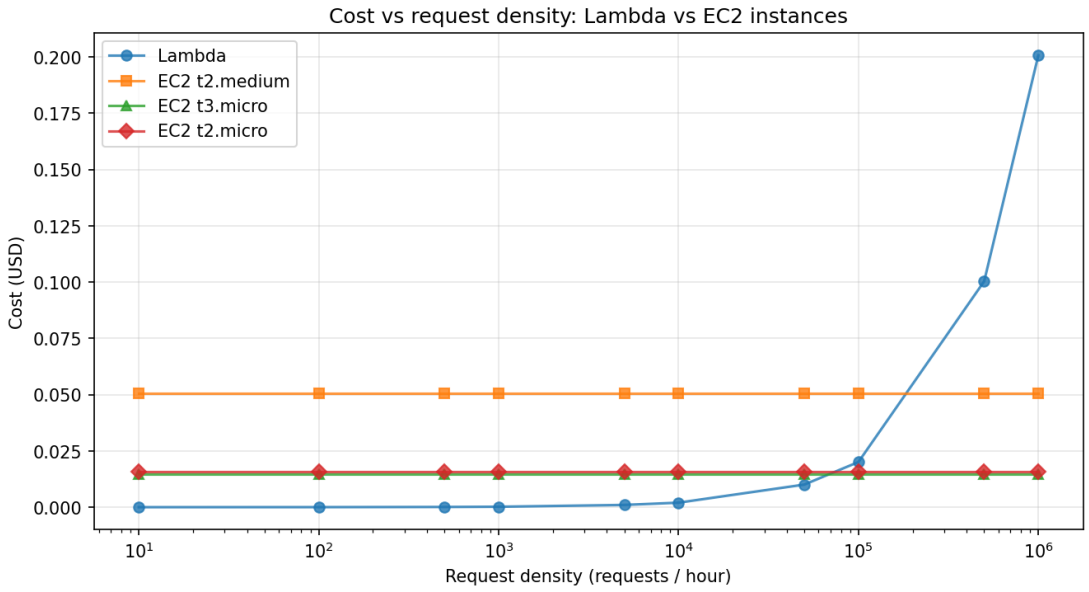

Fig. 7. Predicted relationship between cost and request density for AWS EC2 and Lambda

request/hour). Thus, I conclude that the model inference provider should choose the AWS services based on their demand. If the request density is relatively large, the EC2 instances would be a better choice, where as Lambda function will be more cost-effecient if the request density is much smaller.

However, due to the limited time and resources accessed for AWS academy account, I have to admit the existence of possible flaws and incomplete coverage inside the design of the experiment. The first important missing aspect inside the experiment is the GPU instances. The GPU resource can hugely accelerate the inference process but at a higher cost. Unfortunately, the AWS academy account does not have permission to launch GPU instance for experiment. Future experiment conducted on GPU instance could provide stronger suggestions on performance and cost analysis. The second important flaws inside the experiment design is the fluctuation of request density. Within my project, all requests are sent sequentially within the experiment period, so the request density is only applied to the experiment interval. However, in real situation, the model inference service should always remain accessible by user in any time, and the request density should follow certain distribution with peaks in user working hour and floors in user sleeping hour. I fail to mimic the fluctuation of request density inside this experiment, which might completely changes the current analysis result. Therefore, future research could focus on variate request density circumstance for the comprehensiveness of the experiment.

After filling the flaws I mentioned or other possible imperfections, the comprehensive experiment can provide more meaningful insight to all cloud model inference provider on their current established service, and possible improvement and refinement of their cloud architecture for better inference service for user.

Acknowledgement

I acknowledge the use of Generative AI tools, more specifically Cursor and Gemini, during the development of this project. These tools were applied to assist in coding and debugging tasks. Their contribution helped improve efficiency and resolve issues that I cannot handle effectively, such as writing scripts for environment packages for lambda function.

References

[1] Amazon Web Services, "Amazon EC2 - virtual server cloud computing," https://aws.amazon.com/ec2/, 2026, accessed: March 7, 2026.

[2] ——, "AWS Lambda - serverless compute," https://aws.amazon.com/lambda/, 2026, accessed: March 7, 2026.

[3] R. Bagai, "Comparative analysis of aws model deployment services," *International Journal of Computer Trends and Technology*, vol. 72, no. 5, p. 102–110, May 2024. [Online]. Available: http://dx.doi.org/10.14445/22312803/IJCTT-V72I5P113

[4] S. Agarwal, "Benchmarking serverless efficiency for e-learning platforms: A comparative study of aws lambda and ec2 models," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 23s, p. 3314–3319, Aug. 2024. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/7673

[5] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: https://doi.org/10.24432/C5XW20.

[6] M. Zwitter and M. Soklic, "Breast Cancer," UCI Machine Learning Repository, 1988, DOI: https://doi.org/10.24432/C51P4M.

[7] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. ACM, Aug. 2016, p. 785–794. [Online]. Available: http://dx.doi.org/10.1145/2939672.2939785

[8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

Artifact Appendix

Github Repository: https://github.com/sunnyday9/cs5296project

Here is the artifact of reproducing EC2 and Lambda experiment. Most of the processes are also written inside the demo video and .sh scripts of github repository.

A. EC2 Experiment

Step 1 Create 2 instances on AWS EC2 with one being server and the other being client. Please make sure using Ubuntu 22.04 Server, a security group supporting all kinds of connections (eg. SSH, SMTP, TCP, UDP etrc.), and 30GB storage for both instances.

Step 2 Open three terminal using the MobaXterm with two terminal of server (one for inference service and one for resources usage monitor), and one terminal of client for inference requests.

Step 3 Both server and client needs environment setup, the setup process are given in scripts/setup_ec2.sh

Step 4 Conduct the experiment by following

- Start server model inference service using "python3 ec2/server.py –host 0.0.0.0 –port 5000 –model-type xgb –dataset adult" (sample argument)
- Start server resource usage monitor using "python3 scripts/resource_logger.py –interval 1 –output results/resource_ec2_xgb_n100.csv" (sample argument)
- Start client request using "python3 scripts/client_ec2.py –url http://<EC2_PUBLIC_IP>:5000 –num-requests 100 –data data/test_inputs.npy –output results/ec2_latencies.csv" (sample argument), and change the EC2_PUBLIC_IP with the server public IP address.

Step 5 Finally, we could copy both inference and resource usage results back to the local computer using "scp -i /path/to/your-key.pem ubuntu@<EC2_PUBLIC_IP >: /cs5296project/results/path/to/file /path/to/local/destination" for further analysis.

B. Lambda Experiment

Step 1 Create Lambda function using the Python3.11 version and LabRole as the lambda function custom role.

Step 2 Upload project_xgb.zip (or project_mlp.zip) to the Amazon S3 bucket and loading the zip file into the Lambda function using URL.

Step 3 Setting configurations for the lambda function in the following list

- Runtime handler function: handler.lambda_handler
- General Configuration: Memory = 128MB, Timeout = 3min
- Environment Variable: (DATASET, cancer/adult) for (key, value) pair.

And then make a basic testing to ensure the successfully inference service of Lambda function.

Step 4 Open a terminal on local computer and setting /.aws/credentials be the AWS CLI credential offered in AWS Academy. And setting up environment based on the scripts/experiment_lambda.sh

Step 5 Finally, Conduct the experiment by following

- Running inference service using "python3 scripts/client_lambda.py –function-name project-mlp –num-requests 1000 –data data/cancer_test_inputs.npy –output results/lambda_latencies.csv –region us-east-1" (sample argument)
- Running script gathering cloudwatch information using "python3 scripts/fetch_lambda_metrics.py –function-name project-mlp –region us-east-1 –start "2026-02-22 14:35:00" –end "2026-02-22 15:40:00" –output results/lambda_mlp_cancer_metrics.json" (sample argument)

and all the result will be under results folder.

C. Analysis

After gathering all the result data on the local computer, following /scripts/result_analysis/analysis.sh to get the plot output and data summary.